

ICS EXE 2

2025.03.13

Problem 1

The code below represents a simple GPU computing task that simulates a 3D mean filter. The new value of each voxel is equal to the average of itself and its 26 surrounding neighbors (a 3x3x3 region). This is commonly used for noise removal in 3D medical images.

```
1. #include <cuda_runtime.h>
2. #include <stdio.h>
3. #include <math.h>
4.
5. typedef float Voxel;
6.
7. __global__ void smooth3D(Voxel *input, Voxel *output, int width, int height, int
depth) {
8.     int x = _____
9.     int y = _____
10.    int z = _____
11.
12.    float sum = 0.0f;
13.    int count = 0;
14.    for (int dz = -1; dz <= 1; dz++) {
15.        for (int dy = -1; dy <= 1; dy++) {
16.            for (int dx = -1; dx <= 1; dx++) {
17.                int nx = x + dx;
18.                int ny = y + dy;
19.                int nz = z + dz;
20.                if (nx >= 0 && nx < width && ny >= 0 && ny < height && nz >= 0 && nz <
depth) {
21.                    int idx = nz * (width * height) + ny * width + nx;
22.                    sum += input[idx];
23.                    count++;
24.                }
25.            }
26.        }
27.    }
28.    int outIdx = z * (width * height) + y * width + x;
29.    output[outIdx] = sum / count;
30. }
31. int main() {
```

```

32.     int width = 64;
33.     int height = 64;
34.     int depth = 64;
35.     size_t numVoxels = width * height * depth;
36.     size_t memSize = numVoxels * sizeof(Voxel);
37.
38.     Voxel *h_input = (Voxel*)malloc(memSize);
39.     Voxel *h_output = (Voxel*)malloc(memSize);
40.
41.     for (int z = 0; z < depth; z++) {
42.         for (int y = 0; y < height; y++) {
43.             for (int x = 0; x < width; x++) {
44.                 int idx = z * (width * height) + y * width + x;
45.                 float dx = x - width / 2.0f;
46.                 float dy = y - height / 2.0f;
47.                 float dz = z - depth / 2.0f;
48.                 h_input[idx] = (dx * dx + dy * dy + dz * dz < 100.0f) ? 1.0f : 0.0f;
49.             }
50.         }
51.     }
52.
53.     Voxel *d_input, *d_output;
54.     cudaMalloc(&d_input, memSize);
55.     cudaMalloc(&d_output, memSize);
56.     cudaMemcpy(d_input, h_input, memSize, cudaMemcpyHostToDevice);
57.
58.     dim3 blockDim(8, 8, 8);
59.     dim3 gridDim(______);
60. );
61. smooth3D<<<gridDim, blockDim>>>(d_input, d_output, width, height, depth);
62. if (cudaGetLastError() != cudaSuccess) {
63.     printf("CUDA Launch Error\n");
64.     return 1;
65. }
66. cudaDeviceSynchronize();
67. cudaMemcpy(h_output, d_output, memSize, cudaMemcpyDeviceToHost);
68. cudaFree(d_input);
69. cudaFree(d_output);
70. free(h_input);
71. free(h_output);
72.
73. return 0;
74. }
75.

```


Problem 2

```
1. #define BUCKET_COUNT 1024
2. struct Node {
3.     char *word;
4.     int count;
5.     struct Node *next;
6. };
7. struct Node *hash_table[BUCKET_COUNT];
8. int get_length(char *s) {
9.     int len = 0;
10.    while (*s++) len++;
11.    return len;
12. }
13. Node* find_or_insert_in_bucket(Node* head, char* token, int token_len) {
14.    Node* curr = head;
15.    while (curr != NULL) {
16.        if (get_length(curr->word) == token_len && strcmp(curr->word, token) == 0) {
17.            curr->count++;
18.            return curr;
19.        }
20.        curr = curr->next;
21.    }
22.    Node* new_node = (Node*)malloc(sizeof(Node));
23.    new_node->word = strdup(token);
24.    new_node->count = 1;
25.    new_node->next = head;
26.    return new_node;
27. }
28. void process_file(char **lines, int line_count) {
29.    for (int i = 0; i < line_count; i++) {
30.        char *line = lines[i];
31.        char *token = strtok(line, " \n");
32.        while (token != NULL) {
33.            int len = get_length(token);
34.            int index = hash_function(token, len) % BUCKET_COUNT;
35.            Node* result = find_or_insert_in_bucket(hash_table[index], token, len);
36.            if (result->count == 1 && hash_table[index] != result) {
37.                hash_table[index] = result;
38.            }
39.            token = strtok(NULL, " \n");
40.        }
41.    }
42. }
```

The partial gprof report results for the above code are as follows. Please carefully read the code and performance analysis results to answer the following questions.

% time	cumulative seconds	self seconds	calls	function name
68.5%	3.42	3.42	1,000,000	find_or_insert_in_bucket
16.5%	4.25	0.83	15,000,000	get_length
9.0%	4.70	0.45	1,000,000	hash_function
6.0%	5.00	0.30	200,000	malloc

1. Based on the gprof results, which function should we prioritize for performance optimization, and what are the possible optimization strategies?
2. If **BUCKET_COUNT** is set to 16384, what will be the speedup according to Amdahl's Law? (Assume the hash function distributes data uniformly and the function's execution time is proportional to the average linked list length.)